GPUMODE Lecture Study Notes: Lecture 53
torch.compile Q&A

Core Problem the Lecture Addresses

This lecture is a question-and-answer session on `torch.compile`, led by Richard Zou from the PyTorch compiler team. The lecture addresses a practical systems problem:

> How should engineers reason about `torch.compile` as a real compiler for PyTorch programs, especially when performance, graph breaks, custom operators, Triton kernels, recompilation, graph inspection, training, and deployment all interact?

The talk is not a high-level overview of PyTorch 2. It is a deep practical discussion of the failure modes and design philosophy of `torch.compile`. The key theme is that `torch.compile` can produce strong baseline performance when it can see enough of the computation, but its performance depends heavily on graph capture quality, operator visibility, shape stability, backend code generation, and runtime configuration.

The compiler pipeline can be summarized as:

$$\text{Python program} \xrightarrow{\text{Dynamo}} \text{Torch graph} \xrightarrow{\text{AOTAutograd / AOTDispatcher}} \text{normalized graph} \xrightarrow{\text{Inductor}} \text{generated ke}$$

The main engineering questions are:

- Why does `torch.compile` sometimes speed up code dramatically and sometimes not?
- What causes graph breaks, and why do graph breaks limit fusion?
- How should custom CUDA, C++, Python, Pillow, NumPy, or Triton kernels be integrated?
- What does `torch.compile` treat as a black box?
- What optimizations does the compiler perform automatically?

- How can one inspect generated graphs and generated code?
- Why can compilation time be large?
- How does compilation differ between inference and training?
- What is the practical deployment path for C++ or AOT inference?
- How can engineers contribute to PyTorch compiler development?

A concise mental model is:

> `torch.compile` is useful when it can convert a large enough region of PyTorch tensor computation into an optimizable graph. The more visible, stable, and tensor-centric the computation is, the more opportunity the compiler has to fuse, specialize, autotune, and lower it efficiently.

Required Background

PyTorch Eager Execution

In ordinary PyTorch eager mode, each operation executes immediately. For example:

```
import torch

x = torch.randn(1024, device="cuda")
y = torch.sin(x)
z = torch.cos(y)
```

In eager execution, `torch.sin` and `torch.cos` may each dispatch separate operators and launch separate CUDA kernels. The runtime cost can be approximated as:

$$T_{\text{eager}} \approx T_{\text{launch},1} + T_{\text{kernel},1} + T_{\text{launch},2} + T_{\text{kernel},2}.$$

Eager mode is flexible and debuggable, but it often leaves performance on the table because it cannot automatically optimize across Python-level operation boundaries.

`torch.compile`

`torch.compile` is PyTorch's just-in-time compilation interface:

```
import torch

def f(x):
    y = torch.sin(x)
```

```
    z = torch.cos(y)
    return z

compiled_f = torch.compile(f)
out = compiled_f(torch.randn(1024, device="cuda"))
```

The compiler tries to capture tensor operations into a graph, transform the graph, and lower it into efficient generated code.

The compiler stack discussed in the lecture has three major components:

`torch.compile` = Dynamo + AOTAutograd / AOTDispatcher + Inductor.

- **Dynamo** captures Python bytecode into a graph of PyTorch operations.
- **AOTAutograd / AOTDispatcher** normalizes, functionalizes, and handles autograd-related graph transformations.
- **Inductor** lowers the normalized graph to generated code, often Triton kernels on NVIDIA GPUs and C++ kernels on CPU.

Graph Capture

A graph is a structured representation of tensor operations and their dependencies. For a function such as:

```
def f(x):
    y = torch.sin(x)
    z = torch.cos(y)
    return z
```

the graph can be represented conceptually as:

$$x \to \sin(x) \to \cos(\sin(x)) \to z.$$

Once computation is represented as a graph, the compiler can perform transformations such as:

- dead code elimination;
- common subexpression elimination;
- operator normalization;
- functionalization;
- pattern matching;
- pointwise fusion;
- reduction fusion;

- epilogue fusion;
- memory planning;
- kernel selection;
- autotuning.

Graph Breaks

A graph break occurs when Dynamo reaches Python behavior that it cannot safely capture. Consider:

```
def f(x):
    y = torch.sin(x)
    print(y)
    z = torch.cos(y)
    return z
```

The `print` statement is not ordinary tensor computation. Dynamo may capture the first region:

$$G_1 : x \mapsto \sin(x),$$

then run the `print` outside the graph, and then capture a second region:

$$G_2 : y \mapsto \cos(y).$$

The resulting execution is:

$$\text{compiled graph } G_1 \to \text{Python side effect} \to \text{compiled graph } G_2.$$

This matters because the compiler can no longer fuse operations across the break. Without the graph break, `sin` and `cos` might be fused into a single kernel. With the graph break, they live in separate compiler regions.

Kernel Launch Overhead

A CUDA kernel launch has nonzero overhead. If a program launches many small kernels, total execution time may be dominated by launch cost:

$$T_{\text{total}} = \sum_{i=1}^{N} \left(T_{\text{launch},i} + T_{\text{kernel},i} \right).$$

Fusion reduces the number of launches. CUDA graphs reduce repeated launch overhead by recording and replaying a fixed sequence of GPU work.

Arithmetic Intensity and the Roofline Model

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

A kernel is approximately memory-bound when:

$$I < \frac{P_{\text{peak}}}{B_{\text{peak}}},$$

where P_{peak} is peak compute throughput and B_{peak} is peak memory bandwidth.

Pointwise kernels often have low arithmetic intensity. For example:

$$y_i = \sin(x_i), \qquad z_i = \cos(y_i).$$

Without fusion, global memory traffic includes:

$$\text{read } x + \text{write } y + \text{read } y + \text{write } z.$$

With fusion, y_i can stay in a register:

$$\text{read } x + \text{write } z.$$

Thus fusion improves performance by reducing both memory traffic and launch overhead.

Main Concepts

Common Performance Pitfalls of `torch.compile`

The lecture begins with a practical question: what are common performance pitfalls of `torch.compile`, and how should one write code that is easy for the compiler to optimize?

The pitfalls fall into three categories:

1. front-end graph capture problems;
2. backend tuning and code-generation configuration;
3. recompilation and shape specialization issues.

Pitfall 1: Too Many Graph Breaks

Graph breaks are one of the most important front-end performance problems. They introduce overhead and prevent optimizations across graph boundaries.

Example:

```
def f(x):
    y = torch.sin(x)
    print(y)       # Graph break
    z = torch.cos(y)
    return z

compiled_f = torch.compile(f)
```

The compiler may generate two separate subgraphs:

$$G_1(x) = \sin(x), \qquad G_2(y) = \cos(y).$$

Instead of one fused region:

$$G(x) = \cos(\sin(x)).$$

The performance cost is not merely the Python print. The deeper cost is that optimization visibility is lost. If the compiler cannot see both sides of the operation together, it cannot fuse them into one generated kernel.

To debug graph breaks, use:

```
compiled_f = torch.compile(f, fullgraph=True)
```

With `fullgraph=True`, `torch.compile` errors when it encounters a graph break. This helps locate unsupported code. However, the lecture emphasizes that `fullgraph=True` can be difficult to use on real-world models because realistic models may contain graph breaks that are not easy to remove immediately.

Pitfall 2: Custom Kernels Hide Computation

If most of a function consists of custom kernels, `torch.compile` may have very little to optimize.

Example:

```
def f(x, y):
    a = custom_cuda_op_1(x)
    b = custom_cuda_op_2(a, y)
    c = custom_cuda_op_3(b)
    return c
```

From the compiler's perspective, each custom operation is opaque unless it is represented with proper PyTorch operator metadata. The compiler generally cannot inspect arbitrary CUDA C++ source and rewrite it.

A better structure, when practical, is to expose more computation as native PyTorch operations:

$$\text{opaque custom kernel} \quad \longrightarrow \quad \text{visible PyTorch tensor operations.}$$

This gives Dynamo and Inductor a graph that can be optimized.

Pitfall 3: Not Using Backend Modes

The lecture mentions `mode="reduce-overhead"`:

```
compiled_f = torch.compile(f, mode="reduce-overhead")
```

On NVIDIA GPUs, this can activate CUDA graphs. CUDA graphs record GPU work once and replay it on future invocations. The approximate model is:

$$T_{\text{first run}} = T_{\text{compile}} + T_{\text{capture}} + T_{\text{execute}},$$

while future runs become:

$$T_{\text{future run}} = T_{\text{replay}}.$$

This is useful when launch overhead is a significant part of runtime.

Pitfall 4: Autotuning Cost

Inductor can autotune generated kernels. Autotuning means generating multiple candidate implementations, benchmarking them, and choosing the fastest one.

The first-use cost is:

$$T_{\text{first use}} = T_{\text{compile}} + T_{\text{candidate generation}} + T_{\text{benchmarking}} + T_{\text{selected execution}}.$$

The steady-state cost is:

$$T_{\text{steady state}} = T_{\text{selected execution}}.$$

This trade-off is valuable for long-running training or inference workloads, but it may be too expensive for short scripts or frequently changing input shapes.

Pitfall 5: Recompilation

`torch.compile` may recompile when assumptions change. Common reasons include:

- tensor shapes change;
- Python integer arguments change;
- dtype changes;
- device changes;
- layout or stride patterns change;
- control flow changes;
- guards generated during tracing are invalidated.

The compiler specializes based on assumptions. These assumptions can be represented as a guard predicate:

$$\mathcal{G}(\text{inputs}, \text{locals}, \text{globals}) = \text{true}.$$

If $\mathcal{G}$ fails, the compiled graph may no longer be valid, and recompilation can occur.

For example, if a graph was compiled for input shape:

$$x \in \mathbb{R}^{B \times N},$$

and later receives:

$$x' \in \mathbb{R}^{B' \times N},$$

then the compiler may need a new specialization unless dynamic shapes are enabled and supported for the region.

How to Write Compiler-Friendly PyTorch

A compiler-friendly function has:

- large straight-line tensor regions;
- few unsupported Python side effects;
- stable tensor shapes when possible;
- native PyTorch operations rather than opaque custom kernels;
- explicit metadata for custom operators;

- limited Python scalar logic inside the hot path;
- minimal mutation unless the compiler can functionalize it.

A good design principle is:

Python orchestration outside compiled regions, tensor math inside compiled regions.

Custom Kernels and Custom Operators

Kernel Versus Operator

The lecture distinguishes between a kernel and an operator.

A **kernel** is low-level code that performs computation over raw data pointers. Examples include:

- CUDA C++ kernels;
- Triton kernels;
- C++ kernels;
- low-level Pillow routines;
- NumPy native routines.

An **operator** is a kernel plus PyTorch-facing metadata. The metadata tells PyTorch how the computation behaves.

The operator must answer questions such as:

- What is the operator's schema?
- What inputs does it take?
- What outputs does it produce?
- Does it mutate any inputs?
- What are the output shapes and dtypes?
- How should it behave under fake tensor or meta execution?
- Does it participate in autograd?

Why Custom Operators Matter

A raw CUDA kernel can be called from Python, but PyTorch may not know what it does. This is a problem for:

- autograd;
- fake tensor propagation;

- symbolic shape propagation;
- functionalization;
- compilation;
- export;
- graph transformation.

For `torch.compile`, the key requirement is that the compiler must be able to trace the program without performing the real computation. Therefore, it needs a fake or meta implementation that describes output metadata.

Python Custom Operator Example

The lecture gives a conceptual example using a Pillow crop operation. A simplified version is:

```
import torch
from PIL import Image
import numpy as np

def crop_impl(x, box):
    image = Image.fromarray(x.cpu().numpy())
    cropped = image.crop(box)
    arr = np.asarray(cropped)
    return torch.from_numpy(arr).to(x.device)
```

As plain Python, this may work in eager mode, but the compiler cannot infer output metadata unless it is told how the output shape depends on the input and crop box.

A custom operator wrapper conceptually looks like:

```
import torch

@torch.library.custom_op("mylib::crop", mutates_args=())
def crop(x: torch.Tensor, box: tuple) -> torch.Tensor:
    return crop_impl(x, box)
```

The missing piece for compilation is a fake or meta kernel:

$$(x, \text{box}) \mapsto \text{output metadata}.$$

If the crop box is:

$$\text{box} = (\ell, u, r, d),$$

then the output height and width are:

$$H_{\text{out}} = d - u, \qquad W_{\text{out}} = r - \ell.$$

A fake implementation might conceptually be:

```
@crop.register_fake
def crop_fake(x, box):
    left, upper, right, lower = box
    out_h = lower - upper
    out_w = right - left
    channels = x.shape[2]
    return torch.empty(
        (out_h, out_w, channels),
        device=x.device,
        dtype=x.dtype
    )
```

The actual API details may differ by PyTorch version, but the idea is stable: the compiler needs metadata propagation.

C++ Custom Operators

Custom operators can also be registered in C++. The conceptual structure is:

```
TORCH_LIBRARY(myops, m) {
    m.def("fast_crop(Tensor x, int left, int upper, int right, int lower) -> Tensor");
}

TORCH_LIBRARY_IMPL(myops, CUDA, m) {
    m.impl("fast_crop", fast_crop_cuda);
}

TORCH_LIBRARY_IMPL(myops, Meta, m) {
    m.impl("fast_crop", fast_crop_meta);
}
```

The CUDA implementation performs the actual computation. The meta implementation returns a tensor-like metadata object with the expected shape, dtype, and device semantics.

Black-Box Treatment of Custom Operators

When `torch.compile` sees a custom operator, it treats it as a black box. It does not reach into the CUDA code and optimize the internal implementation.

This has two consequences:

1. The compiler can schedule the custom operator as a node in the graph.

2. The compiler generally cannot fuse into or out of the internals of that custom operator.

The black-box model is sometimes a limitation and sometimes a feature. It is a limitation when the user wants fusion across the custom kernel. It is a feature when the custom code is difficult for Dynamo to understand; wrapping it as a custom operator can provide an escape hatch.

Why Black Boxes Are Hard to Optimize

Suppose a custom CUDA kernel computes:

$$z_i = \cos(\sin(x_i)).$$

If written in PyTorch, the compiler sees:

$$x \to \sin(x) \to \cos(\cdot) \to z.$$

If hidden inside a custom CUDA kernel, the compiler sees only:

$$x \to \text{custom\_op}(x) \to z.$$

The compiler does not know whether the custom operation is pointwise, reduction-like, memory-bound, compute-bound, associative, fusible, or mutating. Without semantic knowledge, safe optimization is impossible.

Triton Kernels and `torch.compile`

Triton Kernels Usually Work

The lecture explains that user-defined Triton kernels generally work with `torch.compile`. The compiler understands enough about Triton calls to include them in the generated graph. However, the body of the Triton kernel is still mostly treated as a black box.

A simple Triton-style kernel might be:

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
```

```
    y = tl.load(y_ptr + offsets, mask=mask)
    out = x + y
    tl.store(out_ptr + offsets, out, mask=mask)
```

The compiler may represent the call to this Triton kernel as a node and lower it appropriately.

Why Triton Requires Some Special Handling

The lecture clarifies a subtle point: if Triton kernels are black boxes, why does `torch.compile` need to keep up with Triton features?

The answer is that the body of the Triton kernel may be black-box-like, but the surrounding Python-side Triton APIs are visible to Dynamo and Inductor. Examples include:

- host-side Tensor Memory Accelerator setup;
- Triton autotuning decorators;
- launch metadata;
- mutation analysis;
- pointer aliasing behavior;
- functionalization requirements.

For example, some Triton features involve Python code outside the kernel body. Dynamo sees that Python code, so `torch.compile` must understand or lower it.

Autotuning Triton Kernels

Triton has its own autotuning mechanism. Inductor also has an autotuning infrastructure. The lecture explains that PyTorch did not simply copy and paste the Triton autotuning decorator into generated code. Instead, it had to integrate Triton autotuning with Inductor's autotuning system.

The abstract autotuning problem is:

$$k^\star = \arg\min_{k \in \mathcal{K}} T(k),$$

where $\mathcal{K}$ is the set of candidate kernel configurations and $T(k)$ is measured runtime for candidate k.

Candidate configurations might vary:

- block size;
- number of warps;
- pipeline stages;

- tiling dimensions;
- memory layout;
- use of tensor cores;
- scheduling strategy.

Mutation Analysis for Triton

A Triton kernel receives pointers. Not every pointer is written to. For example:

```
@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)
```

Here:

$$x_{\text{ptr}} \text{ is read}, \qquad y_{\text{ptr}} \text{ is read}, \qquad \text{out}_{\text{ptr}} \text{ is written}.$$

The compiler needs to know which inputs are mutated because functionalization depends on mutation semantics.

If the compiler cannot determine which pointers are written, it may conservatively assume that all pointer arguments are mutated. That conservative assumption can cause problems for functionalization and later defunctionalization.

Functionalization

Functionalization converts mutating operations into non-mutating equivalents when possible. For example, an in-place update:

$$x \leftarrow x + 1$$

can be represented functionally as:

$$x_{\text{new}} = x_{\text{old}} + 1.$$

This is useful because pure functional graphs are easier to transform, differentiate, partition, and optimize.

For pointer-based kernels, the compiler must know which pointers represent outputs. Otherwise, it cannot correctly convert mutation into functional graph semantics.

Where `torch.compile` Is Going

Value Proposition

The lecture frames the value proposition of `torch.compile` as follows:

> Users can spend hours, days, or weeks hand-tuning specific kernels and models. `torch.compile` aims to provide a strong baseline automatically, so users do not always need to write custom kernels.

The value of the compiler is proportional to how close it gets to hand-tuned performance:

$$\text{Compiler value} \propto \frac{P_{\text{compiled}}}{P_{\text{hand tuned}}},$$

where P denotes achieved performance.

Continued Performance Improvements

The PyTorch compiler team is continuing to improve generated code performance, especially for matrix multiplication and newer hardware.

Matrix multiplication performance matters because GEMM dominates many deep learning workloads:

$$C = AB, \qquad A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

The FLOP count is:

$$\text{FLOPs}_{\text{GEMM}} = 2MNK.$$

A compiler can choose among different implementations:

$$\mathcal{A} = \{\text{cuBLAS}, \text{CUTLASS}, \text{Triton}, \text{custom generated kernel}, \text{fused variant}\}.$$

The best implementation depends on:

- GPU architecture;
- tensor shapes;
- dtype;
- memory layout;
- whether there is a fused epilogue;

- whether tensor cores can be used;
- batch size;
- alignment;
- surrounding operations.

Better Usability

The lecture emphasizes that `torch.compile` is often perceived as powerful but difficult to understand when it fails.

The team wants to improve:

- error messages;
- graph-break diagnostics;
- escape hatches;
- user control over compilation boundaries;
- ways to inspect what happened;
- ways to recover when a compiler bug appears.

The analogy used is a normal programming language compiler: if a compiler feature causes trouble, a programmer should be able to work around it rather than abandon the entire compiler.

Compilation Time

Compilation time is a major roadmap item. The lecture notes that eager mode has essentially no just-in-time compilation overhead, while `torch.compile` may have substantial first-run overhead.

The first-run cost can be decomposed as:

$$T_{\text{first run}} = T_{\text{Dynamo capture}} + T_{\text{graph transforms}} + T_{\text{AOTAutograd}} + T_{\text{Inductor lowering}} + T_{\text{code generation}} + T_{\text{Triton compilation}} +$$

The steady-state cost is closer to:

$$T_{\text{steady state}} = T_{\text{compiled execution}}.$$

The problem is severe when:

$$T_{\text{compile}} \gg T_{\text{eager execution}}.$$

For example, if eager execution takes 1 second but compilation takes 15 seconds, users may not want compilation unless they run the compiled function many times.

Sources of Compilation Overhead

The lecture mentions several sources:

- Python bytecode tracing;
- Python-level graph construction;
- Python-level optimization passes;
- shape propagation;
- repeated analysis over graph nodes;
- backend transformations;
- code generation;
- Triton compilation;
- autotuning benchmarks.

A key engineering issue is algorithmic complexity. If a compiler pass is quadratic in graph size, then for graph size n:

$$T(n) = O(n^2).$$

This can become catastrophic for large graphs. The goal is to keep major passes close to linear:

$$T(n) = O(n).$$

The lecture notes that some extreme one-hour compilation problems often come from an accidentally quadratic algorithm.

Moving Components to C++

Many compiler components are implemented in Python. This improves development velocity but can cost runtime. Moving tracing infrastructure and hot compiler paths to C++ could provide substantial speedups.

A rough performance model is:

$$T_{\text{Python compiler}} = N_{\text{ops}} \cdot t_{\text{Python per op}},$$

whereas a lower-level implementation might reduce the constant:

$$T_{\text{C++ compiler}} = N_{\text{ops}} \cdot t_{\text{C++ per op}}, \qquad t_{\text{C++ per op}} < t_{\text{Python per op}}.$$

Hardware-Level Explanation

Why Fusion Helps GPUs

GPUs are throughput machines. They need enough parallel work and enough arithmetic intensity to hide latency and use available bandwidth and compute.

For two pointwise operations:

$$y_i = \sin(x_i), \qquad z_i = \cos(y_i),$$

a non-fused execution may launch two kernels. In each kernel, threads load and store global memory.

Non-fused memory traffic per element is approximately:

$$\text{bytes}_{\text{non-fused}} = \text{load } x_i + \text{store } y_i + \text{load } y_i + \text{store } z_i.$$

For FP32, this is roughly:

$$4 + 4 + 4 + 4 = 16 \text{ bytes per element.}$$

Fused execution is:

$$z_i = \cos(\sin(x_i)).$$

Memory traffic per element becomes:

$$\text{bytes}_{\text{fused}} = \text{load } x_i + \text{store } z_i = 4 + 4 = 8 \text{ bytes per element.}$$

Thus fusion can reduce global memory traffic by approximately:

$$\frac{16}{8} = 2.$$

It can also reduce kernel launches by:

$$2 \to 1.$$

Thread-Level Execution for Pointwise Fusion

A fused pointwise kernel might assign one element or several elements to each GPU thread.

Conceptually:

```
__global__ void fused_sin_cos(float* out, const float* x, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float tmp = sinf(x[idx]);
        out[idx] = cosf(tmp);
    }
}
```

The indexing formula is:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

Threads in a warp access consecutive addresses when i is consecutive. This gives coalesced global memory access:

$$x_i, x_{i+1}, \ldots, x_{i+31}.$$

The intermediate value *tmp* lives in a register, not global memory.

CUDA Graphs and Launch Overhead

CUDA graphs help when the same sequence of kernels is repeatedly executed. Instead of launching each kernel individually, the runtime records a graph of work and replays it.

If a sequence contains N kernels, ordinary execution has launch overhead:

$$T_{\text{launch ordinary}} = \sum_{i=1}^{N} T_{\text{launch},i}.$$

With CUDA graphs, replay overhead can be closer to:

$$T_{\text{launch graph}} = T_{\text{replay}},$$

where T_{replay} can be much smaller than the sum of individual launches.

This is why `mode="reduce-overhead"` is useful for workloads where the compiled region has repeated stable execution.

GEMM and Epilogue Fusion

Matrix multiplication is often compute-bound on modern GPUs when shapes are large and tensor cores are used. But operations around GEMM can still cause extra memory traffic.

Consider:

$$Y = \mathrm{ReLU}(XW + b).$$

A naive execution might do:

$$T = XW, \qquad U = T + b, \qquad Y = \max(U, 0).$$

An epilogue-fused GEMM computes the activation during the GEMM output stage:

$$Y_{ij} = \max\left(\sum_{k=1}^{K} X_{ik} W_{kj} + b_j, 0\right).$$

This avoids writing the unfused intermediate T and rereading it for ReLU.

Prologue Fusion

Prologue fusion means fusing operations before a matrix multiplication into the GEMM input path. For example:

$$C = \mathrm{matmul}(\mathrm{cast}(A), B).$$

Instead of materializing:

$$A_{\mathrm{fp32}} = \mathrm{cast}(A_{\mathrm{fp16}}),$$

then multiplying:

$$C = A_{\mathrm{fp32}} B,$$

the compiler could load A_{fp16}, cast internally, and feed the values directly into the matrix multiply.

The lecture notes that prologue fusion was an optimization area under improvement.

Mathematical Reasoning

Compilation Payoff Model

Let:

$$T_{\text{compile}} = \text{one-time compilation cost},$$

$$T_{\text{eager}} = \text{runtime per eager execution},$$

$$T_{\text{compiled}} = \text{runtime per compiled execution}.$$

For N executions, eager total time is:

$$T_{\text{total eager}}(N) = NT_{\text{eager}}.$$

Compiled total time is:

$$T_{\text{total compiled}}(N) = T_{\text{compile}} + NT_{\text{compiled}}.$$

Compilation pays off when:

$$T_{\text{compile}} + NT_{\text{compiled}} < NT_{\text{eager}}.$$

Solving for N:

$$N > \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}}.$$

This explains why `torch.compile` is more attractive for long-running training or serving workloads than for tiny one-off scripts.

Recompilation Cost

If the program recompiles R times, total runtime becomes:

$$T_{\text{total}} = \sum_{r=1}^{R} T_{\text{compile},r} + \sum_{i=1}^{N} T_{\text{execute},i}.$$

If recompilation happens frequently, compilation overhead may dominate.

The goal is to minimize R by stabilizing shapes, using dynamic shapes where appropriate, and avoiding changing Python scalar values that become guards.

Fusion Memory Savings

For a chain of m pointwise operations on an array of n elements, assume each operation reads one tensor and writes one tensor. Without fusion, approximate memory traffic is:

$$B_{\text{unfused}} \approx 2mns,$$

where s is bytes per element.

With full fusion, approximate memory traffic is:

$$B_{\text{fused}} \approx 2ns.$$

The theoretical memory traffic reduction is:

$$\frac{B_{\text{unfused}}}{B_{\text{fused}}} \approx m.$$

This is an idealized upper bound. Real kernels may have additional reads for constants, broadcasting, masks, or multiple inputs.

GEMM FLOP Count

For matrix multiplication:

$$C = AB, \qquad A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

each output element requires K multiply-adds:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} B_{kj}.$$

The total FLOP count is:

$$\text{FLOPs} = 2MNK.$$

If runtime is T, achieved throughput is:

$$P_{\text{achieved}} = \frac{2MNK}{T}.$$

For tensor-core GEMM, performance depends heavily on dtype, shape alignment, tile sizes, memory layout, and fusion.

Autotuning Objective

Autotuning searches over candidate kernels:

$$\mathcal{K} = \{k_1, k_2, \ldots, k_m\}.$$

The selected kernel is:

$$k^\star = \arg\min_{k \in \mathcal{K}} T(k).$$

For matrix multiplication, candidates may vary:

- block tile sizes B_M, B_N, and B_K;
- number of warps;
- pipeline stages;
- use of tensor cores;
- split-K strategy;
- epilogue fusion strategy;
- memory layout assumptions.

Code Walkthrough

Basic `torch.compile`

```
import torch

def f(x):
    y = torch.sin(x)
    z = torch.cos(y)
    return z

compiled_f = torch.compile(f)

x = torch.randn(1024, device="cuda")
out = compiled_f(x)
```

Explanation

The function contains two pointwise tensor operations. Dynamo can capture both into one graph if no unsupported Python appears between them. Inductor can then generate a fused kernel that computes:

$$z_i = \cos(\sin(x_i)).$$

A hardware-friendly implementation loads x_i once, computes the intermediate in a register, and stores z_i once.

Graph Break Example

```
import torch

def f(x):
    y = torch.sin(x)
    print(y)
    z = torch.cos(y)
    return z

compiled_f = torch.compile(f)
```

Explanation

The `print` statement creates a side effect. Dynamo cannot simply place arbitrary Python printing inside the tensor graph. Therefore, the function is split:

$$G_1(x) = \sin(x), \qquad G_2(y) = \cos(y).$$

This prevents fusion across the print.

Detecting Graph Breaks with `fullgraph=True`

```
compiled_f = torch.compile(f, fullgraph=True)
```

Explanation

With `fullgraph=True`, graph breaks become hard errors. This mode is useful for diagnosing exactly where the compiler stops capturing. However, it may be too strict for large production models.

Reducing Overhead with CUDA Graphs

```
compiled_f = torch.compile(f, mode="reduce-overhead")
```

Explanation

This mode can enable CUDA graph capture on NVIDIA GPUs. CUDA graphs are effective when:

- the execution pattern is stable;

- shapes are stable;
- memory addresses are reusable or capturable;
- launch overhead is significant.

Inspecting Generated Code

The lecture recommends using logs:

```
TORCH_LOGS=output_code python script.py
```

This prints generated output code. For a fused pointwise example, one should expect to see a generated kernel containing both the sine and cosine operations.

A conceptual generated kernel is:

```
for (int i = 0; i < n; ++i) {
    out[i] = cos(sin(x[i]));
}
```

On GPU, this loop is parallelized across threads.

Using `tlparse`

The lecture also mentions `tlparse`, a tool for inspecting compiler traces.

A conceptual workflow is:

```
TORCH_TRACE=/tmp/tracedir python script.py
tlparse /tmp/tracedir
```

`tlparse` provides a more human-readable view of artifacts from different compiler stages, such as Dynamo graphs, AOTAutograd graphs, and Inductor output.

Pointwise Fusion Example

```
import torch

def f(x):
    return torch.cos(torch.sin(x))

compiled_f = torch.compile(f)

x = torch.randn(1024, device="cuda")
y = compiled_f(x)
```

Hardware Behavior

Each output element is independent:

$$y_i = \cos(\sin(x_i)).$$

The kernel can assign independent elements to independent GPU threads. Threads in a warp access adjacent memory, giving coalesced loads and stores.

Epilogue Fusion Example

```
import torch
import torch.nn as nn

linear = nn.Linear(4096, 4096).cuda()

def f(x):
    y = linear(x)
    z = torch.relu(y)
    return z

compiled_f = torch.compile(f)
```

The operation is:

$$Z = \text{ReLU}(XW^T + b).$$

An epilogue-fused implementation computes:

$$Z_{ij} = \max\left(\sum_{k=1}^{K} X_{ik} W_{jk} + b_j, 0\right).$$

Why This Is Efficient

The ReLU is applied before writing the output to global memory. This avoids materializing an intermediate output and launching a separate ReLU kernel.

Prologue Fusion Example

```
import torch

def f(x, w):
    x_fp32 = x.float()
    return x_fp32 @ w

compiled_f = torch.compile(f)
```

Here, the cast happens before matrix multiplication:

$$C = \text{cast}(X)W.$$

Prologue fusion would avoid materializing $\text{cast}(X)$ as a separate tensor. Instead, the GEMM kernel would load low-precision values and cast them internally.

The lecture notes that this class of optimization was under active improvement.

Custom Operator Shape Propagation

```
import torch

@torch.library.custom_op("mylib::crop", mutates_args=())
def crop(x: torch.Tensor, box: tuple) -> torch.Tensor:
    raise NotImplementedError("Real implementation omitted")
```

A fake implementation gives output metadata:

```
@crop.register_fake
def crop_fake(x, box):
    left, upper, right, lower = box
    out_h = lower - upper
    out_w = right - left
    channels = x.shape[2]
    return torch.empty(
        (out_h, out_w, channels),
        device=x.device,
        dtype=x.dtype
    )
```

Explanation

The fake implementation does not compute pixels. It tells the compiler that:

$$H_{\text{out}} = \text{lower} - \text{upper}, \qquad W_{\text{out}} = \text{right} - \text{left}.$$

This is enough for shape propagation.

Triton Kernel Example

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
```

```
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)

    out = x + y
    tl.store(out_ptr + offsets, out, mask=mask)
```

Mutation Semantics

The compiler needs to infer:

$$x_{\text{ptr}} : \text{read}, \qquad y_{\text{ptr}} : \text{read}, \qquad \text{out}_{\text{ptr}} : \text{write}.$$

This matters because the compiler must functionalize mutation correctly.

Training Graph Example

```
import torch

def f(x):
    y = torch.sin(x)
    z = torch.cos(y)
    return z.sum()

compiled_f = torch.compile(f)

x = torch.randn(1024, device="cuda", requires_grad=True)
loss = compiled_f(x)
loss.backward()
```

For training, the compiler must reason about both forward and backward graphs.

Forward:

$$y = \sin(x), \qquad z = \cos(y), \qquad L = \sum_i z_i.$$

Backward:

$$\frac{\partial L}{\partial z_i} = 1,$$

$$\frac{\partial z_i}{\partial y_i} = -\sin(y_i),$$

$$\frac{\partial y_i}{\partial x_i} = \cos(x_i).$$

Thus:

$$\frac{\partial L}{\partial x_i} = -\sin(\sin(x_i))\cos(x_i).$$

AOTAutograd is responsible for producing forward and backward graphs that Inductor can compile.

Compiler Design

Dynamo

Dynamo is the front end. It captures Python programs by interpreting Python bytecode. It does not merely trace by running tensor operations; it reads bytecode instructions and simulates execution enough to build a graph.

For a function:

```
def f(x):
    return torch.cos(torch.sin(x))
```

Dynamo observes calls to PyTorch operations and records them into a graph.

The output is a Torch-level graph, sometimes referred to as Torch IR in the lecture.

AOTAutograd / AOTDispatcher

The next stage normalizes and functionalizes the graph. It also handles autograd.

Normalization means converting many possible PyTorch operations into a smaller set of lower-level operations that the backend understands.

If PyTorch has a large operator surface:

$$\mathcal{O}_{\text{PyTorch}} = \{o_1, o_2, \ldots, o_M\},$$

the compiler prefers to lower into a smaller normalized operator set:

$$\mathcal{O}_{\text{normalized}} = \{a_1, a_2, \ldots, a_K\}, \qquad K < M.$$

Functionalization converts mutation-heavy graphs into functional graphs.

For training, AOTAutograd produces both forward and backward graphs:

$$G_{\text{forward}}, \qquad G_{\text{backward}}.$$

Inductor

Inductor is the backend. It takes normalized graphs and generates executable code.

On NVIDIA GPUs, this often includes:

- generated Triton kernels;
- calls to external libraries such as cuBLAS or CUTLASS-like implementations;
- generated C++ wrapper code;
- autotuned kernel choices;
- fused kernels for pointwise, reductions, and epilogues.

On CPU, Inductor may generate C++ vectorized kernels.

Inference Pipeline

For inference, the conceptual flow is:

$$f \to G_{\text{Dynamo}} \to G_{\text{normalized}} \to \text{Inductor code} \to \text{execution}.$$

No backward graph is needed.

Training Pipeline

For training, the conceptual flow is:

$$f \to G_{\text{Dynamo}} \to (G_{\text{forward}}, G_{\text{backward}}) \to \text{Inductor code for forward+Inductor code for backward}.$$

The backward graph may be compiled lazily when `backward()` is actually invoked.

Optimizations Discussed

Dead Code Elimination

Dead code elimination removes computations whose results are unused.

Example:

```
def f(x):
    y = torch.sin(x)
    unused = torch.cos(x)
    return y
```

The operation `torch.cos(x)` is dead because it does not affect the return value.

Mathematically, if the output is:

$$f(x) = \sin(x),$$

then the computation:

$$u = \cos(x)$$

has no path to the output and can be removed.

Common Subexpression Elimination

If the same operation is computed multiple times, the compiler may deduplicate it.

Example:

```
def f(x):
    a = torch.sin(x)
    b = torch.sin(x)
    return a + b
```

This can be rewritten as:

$$a = \sin(x), \qquad \text{return } a + a.$$

This saves computation and possibly memory traffic.

Min-Cut Partitioning and Recomputation

During training, compilers can choose what activations to save and what activations to recompute during backward. This resembles activation checkpointing.

Let:

$$M_{\text{save}} = \text{memory cost of saved activations},$$

$$C_{\text{recompute}} = \text{compute cost of recomputation}.$$

The compiler balances memory and compute:

$$\min\left(\alpha M_{\text{save}} + \beta C_{\text{recompute}}\right),$$

where α and β represent the relative importance of memory and compute time.

Pointwise operations may be cheap to recompute, especially if they can be fused into backward kernels. Therefore, recomputation can reduce memory without much runtime cost.

Pattern Matching

The compiler can recognize known operation patterns and replace them with specialized kernels.

A key example is attention. A naive attention pattern is:

$$S = QK^T,$$

$$P = \text{softmax}\left(\frac{S}{\sqrt{d}}\right),$$

$$O = PV.$$

If the compiler recognizes the pattern, it may replace it with a fused scaled-dot-product attention or FlashAttention-style implementation.

The key idea is:

$$\text{generic graph pattern} \quad \longrightarrow \quad \text{specialized high-performance kernel.}$$

This avoids materializing the full attention matrix when a memory-efficient algorithm is available.

Pointwise and Reduction Fusion

The compiler fuses pointwise operations and some reductions.

Pointwise fusion example:

$$z_i = \cos(\sin(x_i)).$$

Reduction fusion example:

$$s = \sum_i \cos(\sin(x_i)).$$

A fused reduction kernel may compute local values and reduce them within blocks before writing partial or final sums.

Epilogue Fusion

Epilogue fusion attaches operations after a matrix multiplication to the GEMM output stage.

Example:

$$Y = \mathrm{ReLU}(XW + b).$$

Instead of writing $XW + b$ and then launching a ReLU kernel, the GEMM epilogue computes ReLU before storing.

Prologue Fusion

Prologue fusion attaches operations before matrix multiplication to the GEMM input stage.

Example:

$$C = \mathrm{matmul}(\mathrm{cast}(A), B).$$

The cast can be fused into the input load path of GEMM, avoiding a separate materialized cast tensor.

The lecture notes that this optimization was not fully available in the demonstrated case but was coming soon.

Matrix Multiplication Selection

Inductor can choose among multiple matrix multiplication implementations. The decision depends on shape, dtype, layout, hardware, and surrounding operations.

The selection problem is:

$$a^\star = \arg\min_{a \in \mathcal{A}} T(a; M, N, K, \mathrm{dtype}, \mathrm{layout}, \mathrm{hardware}, \mathrm{fusion}),$$

where $\mathcal{A}$ is the set of available algorithms or kernels.

Graph and Code Inspection

`TORCH_LOGS=output_code`

To inspect generated code:

```
TORCH_LOGS=output_code python script.py
```

This is useful for checking whether fusion happened. If two pointwise operations are fused, the generated code should contain both operations in one kernel.

Other Logs

One can also inspect graph-level outputs using other logging modes. A common workflow is to inspect:

- graph breaks;
- Dynamo graph;
- AOTAutograd graph;
- Inductor graph;
- generated output code.

`tlparse`

`tlparse` provides a more structured way to inspect traces. It can display artifacts from multiple compiler stages.

A conceptual workflow is:

```
TORCH_TRACE=/tmp/trace python script.py
tlparse /tmp/trace
```

The lecture demonstrates using `tlparse` to inspect generated graphs and output code.

AOTInductor and C++ Deployment

Inference-Oriented Deployment

The lecture discusses interfacing with compiled models in C++. The recommended path is generally not `torch.compile` directly, but `torch.export` followed by AOTInductor.

The conceptual workflow is:

PyTorch model → `torch.export` → AOTInductor → shared object → C++ runtime.

The output artifact might be a `model.so` that can be loaded from C++.

Why Training Is Harder

AOTInductor is primarily designed for inference artifacts. Training requires a backward graph and optimizer state updates.

Training involves:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t).$$

A deployable training artifact would need:

- forward graph;
- backward graph;
- gradient accumulation behavior;
- optimizer update graph;
- state mutation semantics;
- possibly distributed training semantics.

The lecture states that AOTInductor does not really support training in the same straightforward way because the artifact is optimized for inference and does not expose a normal backward path.

Hacky Forward-Backward Export

The lecture describes a possible hack: build a single graph that explicitly performs both forward and backward.

Conceptually:

```
import torch
import torch.nn.functional as F
from torch.fx.experimental.proxy_tensor import make_fx

def loss_fn(x, weight, target):
    out = F.linear(x, weight)
    loss = F.mse_loss(out, target)
    return loss

def forward_backward(x, weight, target):
    loss = loss_fn(x, weight, target)
    grad_weight, = torch.autograd.grad(loss, (weight,))
    return loss, grad_weight
```

```
example_x = torch.randn(16, 128)
example_w = torch.randn(64, 128, requires_grad=True)
example_t = torch.randn(16, 64)

gm = make_fx(forward_backward)(example_x, example_w, example_t)
```

This explicitly traces a function whose output includes gradients. The resulting graph can potentially be passed into export or lower-level compilation tools.

The limitation is that this is not the normal clean training deployment path. One must also handle optimizer updates and graph partitioning if forward and backward need to be run separately.

Numerical Accuracy Issues

Compiled Versus Eager Numerical Differences

The lecture discusses reports of numerical mismatches between `torch.compile` and eager mode. Such mismatches are important and should be reported.

Floating-point operations are not associative:

$$(a + b) + c \neq a + (b + c)$$

in finite precision.

Compiler transformations may change operation order, fusion boundaries, accumulation precision, or casting behavior. Therefore, small differences can occur.

Compiler May Be More Accurate Than Eager

The lecture notes that sometimes `torch.compile` can be more accurate than eager relative to an FP64 reference. This can happen if the compiler chooses a different accumulation or casting strategy.

Let y_{fp64} be a high-precision reference. Eager error is:

$$e_{\text{eager}} = \|y_{\text{eager}} - y_{\text{fp64}}\| .$$

Compiled error is:

$$e_{\text{compiled}} = \|y_{\text{compiled}} - y_{\text{fp64}}\| .$$

It is possible that:

$$e_{\text{compiled}} < e_{\text{eager}}.$$

However, even improved numerical accuracy can break tests that expect bitwise or near-bitwise eager equivalence.

Debugging Numerical Mismatches

A practical strategy is to shrink the compiled region:

$$\text{large compiled region} \rightarrow \text{smaller compiled region} \rightarrow \text{minimal reproducer}.$$

If the mismatch disappears after excluding a region, that region likely contains the relevant transformation.

Escape Hatches and Usability

Custom Operators as an Escape Hatch

If a piece of code is difficult for Dynamo to capture, one can wrap it as a custom operator. Then `torch.compile` treats it as an opaque node.

This sacrifices internal optimization but can allow the rest of the model to compile.

Non-Strict Tracing

The lecture discusses an emerging escape hatch inspired by JAX-style tracing. The idea is to use a tracing mode that is less dependent on Dynamo's bytecode interpreter and instead records tensor operations called on inputs.

A conceptual example is:

```
def f(x):
    y = unsupported_python_logic(x)
    return torch.cos(torch.sin(y))
```

If Dynamo cannot handle part of the function, a non-strict tracing mode could focus on tensor operations that occur during execution. This is less safe in some ways, but easier to reason about for certain user code.

Selective Graph Break Control

The lecture describes a desired programming model where users can use something like `fullgraph=True` for most of the code, but explicitly allow graph breaks in chosen regions.

The goal is to make compilation boundaries visible in source code:

$$\text{known compiled region} \rightarrow \text{explicit allowed break} \rightarrow \text{known compiled region}.$$

This is better than silently discovering graph breaks only after inspecting logs.

Cache Escape Hatches

The lecture also mentions caching internals: some operations are considered cacheable and others non-cacheable. Bugs in these classifications can affect performance. Better escape hatches would let users or developers override these classifications when needed.

Contribution Pathways

How to Get Involved

The lecture recommends contributing through PyTorch GitHub issues. Useful labels include:

- good first issue;
- actionable;
- high priority;
- PT2-related labels for compiler issues.

The practical advice is to find an issue, attempt a fix, and start a discussion on GitHub. It is not always necessary to ask permission before working on an open issue.

Compiler Contributions Versus Kernel Contributions

The lecture notes that PyTorch compiler work today often needs improvements to compiler infrastructure more than hand-written kernels. Valuable work includes:

- fixing graph breaks;
- improving error messages;
- improving compile-time performance;
- fixing quadratic algorithms;
- improving shape propagation;
- improving functionalization;
- adding tests;

- improving operator support;
- improving documentation;
- improving trace inspection tools.

Performance Intuition

`torch.compile` Is Best When It Sees Large Tensor Regions

The compiler needs visibility. A large graph of native PyTorch tensor operations gives Inductor room to optimize.

Good:

```
def f(x, w, b):
    y = x @ w
    y = y + b
    y = torch.relu(y)
    return y
```

Less ideal:

```
def f(x, w, b):
    y = custom_matmul(x, w)
    y = custom_bias(y, b)
    y = custom_relu(y)
    return y
```

The second version may already be fast if the custom kernels are excellent, but `torch.compile` cannot do much with their internals.

Fusion Helps Most for Memory-Bound Chains

Pointwise chains are often memory-bound. Fusion reduces memory traffic and launch overhead.

For m pointwise operations over n elements, the ideal memory traffic reduction is approximately:

$$\frac{B_{\text{unfused}}}{B_{\text{fused}}} \approx m.$$

This is why fusion is a major compiler optimization.

Autotuning Helps When Steady-State Runtime Matters

Autotuning has upfront cost but improves repeated execution. It is most useful when:

$$N \gg \frac{T_{\text{autotune}}}{T_{\text{baseline}} - T_{\text{tuned}}}.$$

For short workloads, autotuning may not pay off. For long inference serving or training runs, it often does.

Recompilation Can Destroy Performance

If input shapes or Python scalar arguments change frequently, then:

$$R \approx N,$$

where R is number of recompilations and N is number of calls. This is bad because total time becomes dominated by compilation:

$$T_{\text{total}} \approx NT_{\text{compile}} + NT_{\text{execute}}.$$

Stable shapes and guard-friendly code are essential.

Custom Operators Are Both a Tool and a Wall

Custom operators are useful because they allow unsupported code to appear in compiled graphs. But they also form optimization boundaries.

The trade-off is:

$$\text{custom operator} = \text{better encapsulation} - \text{less compiler visibility}.$$

Triton Kernels Are Special Black Boxes

Triton kernels are closer to the compiler ecosystem than arbitrary CUDA kernels. `torch.compile` can often include them, reason about launch structure, and sometimes integrate with autotuning or mutation analysis. But it still does not generally rewrite the internal mathematical body of the Triton kernel.

Common Misconceptions

Misconception: `torch.compile` Optimizes Everything

It only optimizes what it can capture and understand. Unsupported Python, opaque custom kernels, and graph breaks limit optimization.

Misconception: Graph Breaks Are Harmless

Graph breaks may preserve correctness, but they can harm performance by blocking fusion and introducing overhead.

Misconception: A Custom CUDA Kernel Is Always Better

A custom CUDA kernel may be faster for a specific operation, but it may prevent global optimization across surrounding operations. A compiler-visible PyTorch implementation may sometimes win because it enables fusion.

Misconception: Triton Kernels Are Fully Transparent to `torch.compile`

Triton kernels usually work, but the compiler does not fully understand and rewrite arbitrary Triton code. Some surrounding Triton APIs require special support.

Misconception: Compilation Time Is Just Triton Compilation

Compilation time includes Dynamo tracing, Python graph passes, AOTAutograd, shape propagation, Inductor lowering, code generation, Triton compilation, and autotuning.

Misconception: AOTInductor Is a Drop-In Training Export System

AOTInductor is primarily an inference-oriented deployment path. Training requires backward graphs, optimizer state, and mutation semantics that are not provided as a simple default artifact.

Misconception: Numerical Differences Always Mean the Compiler Is Wrong

Numerical differences may come from different operation ordering, fusion, casting, or accumulation. Sometimes compiled output can be closer to an FP64 reference than eager output. Still, unexpected mismatches should be minimized and reported.

Practical Takeaways

1. Use `torch.compile` on large tensor-heavy regions, not on tiny Python-heavy functions.
2. Remove avoidable graph breaks such as `print` statements from hot paths.

3. Use `fullgraph=True` to locate graph breaks when debugging.
4. Inspect generated code with `TORCH_LOGS=output_code`.
5. Use `tlparse` for more readable compiler traces.
6. Avoid frequent recompilation by stabilizing input shapes and Python scalar arguments.
7. Consider dynamic shapes when shape variation is unavoidable.
8. Use `mode="reduce-overhead"` when launch overhead matters and CUDA graph constraints are satisfied.
9. Remember that custom operators are opaque to compiler internals.
10. Provide fake or meta kernels for custom operators so shape propagation works.
11. Prefer native PyTorch operations when compiler fusion may beat manually separated custom kernels.
12. Triton kernels usually work with `torch.compile`, but advanced Triton features may require compiler support.
13. For C++ inference deployment, consider `torch.export` plus AOTInductor rather than ordinary `torch.compile`.
14. Treat compilation time as part of the performance model.

Project or Experiment Ideas

Experiment 1: Graph Break Cost

Write two versions of a function:

```
def no_break(x):
    return torch.cos(torch.sin(x))

def with_break(x):
    y = torch.sin(x)
    print("debug")
    return torch.cos(y)
```

Compile both and measure runtime. Inspect graph breaks with:

```
torch.compile(with_break, fullgraph=True)
```

Expected lesson: graph breaks block fusion and add overhead.

Experiment 2: Pointwise Fusion

Benchmark:

$$y = \cos(\sin(\exp(x))).$$

Compare eager and compiled versions. Inspect generated code using:

```
TORCH_LOGS=output_code python pointwise.py
```

Expected lesson: compiled code should fuse the pointwise chain.

Experiment 3: Recompilation From Shape Changes

Call a compiled function with many different shapes:

```
@torch.compile
def f(x):
    return torch.sin(x) + 1

for n in [128, 256, 512, 128, 1024]:
    x = torch.randn(n, device="cuda")
    y = f(x)
```

Inspect recompilation logs. Then try dynamic-shape options and compare behavior.

Experiment 4: Epilogue Fusion

Compile a linear layer followed by ReLU:

$$Y = \mathrm{ReLU}(XW^T + b).$$

Inspect whether the ReLU appears inside the generated matmul epilogue.

Experiment 5: Custom Operator Boundary

Create a custom operator that wraps a simple pointwise operation and compare:

- native PyTorch implementation;
- custom operator implementation;
- compiled native PyTorch;
- compiled custom operator.

Expected lesson: the custom operator may block internal fusion.

Experiment 6: Triton Kernel Integration

Write a simple Triton add kernel and call it from a compiled function. Test whether `torch.compile` can include the Triton kernel call. Then inspect generated code and mutation behavior.

Experiment 7: Compile-Time Versus Runtime Payoff

Measure:

$$T_{\text{compile}}, \qquad T_{\text{eager}}, \qquad T_{\text{compiled}}.$$

Compute break-even execution count:

$$N^\star = \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}}.$$

This gives a practical answer to whether compilation is worthwhile for the workload.

Final Mental Model

`torch.compile` is a visibility-driven optimizing compiler for PyTorch programs. Dynamo tries to capture Python bytecode into tensor graphs. AOTAutograd and AOTDispatcher normalize, functionalize, and split training graphs into forward and backward computations. Inductor lowers these graphs into generated kernels and library calls, often using Triton on NVIDIA GPUs.

The compiler is powerful when it sees a large, stable, tensor-centric region. It can fuse pointwise operations, fuse reductions, fuse epilogues into matrix multiplications, pattern-match known structures such as attention, select among matrix multiplication implementations, use CUDA graphs to reduce launch overhead, and autotune generated kernels.

Its main enemies are graph breaks, opaque custom kernels, unstable shapes, frequent recompilation, unsupported Python features, excessive compilation overhead, and hidden mutation. Custom operators and non-strict tracing are escape hatches, but they trade compiler visibility for usability.

The best engineer's mental model is:

> Write PyTorch so the compiler can see the computation. Keep Python orchestration outside hot compiled regions. Make tensor math visible, shapes stable, mutation explicit, and graph breaks intentional. Then inspect the generated graphs and code rather than guessing what happened.

Visual Concept

Draw a pipeline from Python code to Dynamo graph capture, then to AOTAutograd normalization and forward/backward splitting, then to Inductor code generation. Under Inductor, draw branches to Triton kernels, C++ kernels, library GEMMs, CUDA graphs, and autotuning. Mark graph breaks as cracks that split the pipeline into separate graph islands.